

|      |            |                   |      |         |
|------|------------|-------------------|------|---------|
| 保密等级 | A          | 泰芯 TXW81x 低功耗开发指南 | 文件编号 | TX-0000 |
| 发行日期 | 2025-04-09 |                   | 文件版本 | V1.0    |

泰芯 TXW81x 低功耗开发指南



|                                                                                     |                                                    |                         |
|-------------------------------------------------------------------------------------|----------------------------------------------------|-------------------------|
|  | 珠海泰芯半导体有限公司<br>Zhuhai Taixin Semiconductor Co.,Ltd | 珠海市高新区港湾一号科创园港 11 栋 3 楼 |
|-------------------------------------------------------------------------------------|----------------------------------------------------|-------------------------|

|      |            |                   |      |         |
|------|------------|-------------------|------|---------|
| 保密等级 | A          | 泰芯 TXW81x 低功耗开发指南 | 文件编号 | TX-0000 |
| 发行日期 | 2025-04-09 |                   | 文件版本 | V1.0    |

## 责任与版权

### 责任限制

本文档内容仅供参考。珠海泰芯半导体有限公司（以下简称“泰芯”）未对本文档信息的准确性或完整性作出任何明示或暗示的陈述或保证，且不对使用本文件信息所产生的后果承担任何责任。

泰芯保留在不另行通知对本文件进行修改、改进或其他变更的权利。

泰芯没有为客户产品的应用、设计验证提供协助的义务。客户应自行对其应用的设计、验证和测试负责，并确保其应用符合所有法律、法规及安全相关的要求。

如因客户未遵守本协议导致任何损害、费用、损失及/或责任，泰芯不承担任何责任；同时，客户应全额赔偿因客户未遵守本协议而给泰芯造成的任何损害、费用、损失及/或责任。

### 版权声明

未经泰芯书面同意，任何一方不得为商业目的修改、改编、变更、翻译或基于本文件创建衍生作品。

未经泰芯书面同意，任何一方不得向第三方披露或分发本文件中提及的任何部分或全部源代码、SDK、二进制文件和目标代码。

任何一方不得修改、反向工程、反汇编、反编译或以其他方式试图发掘任何非源代码部分的 SDK，包括但不限于预编译的二进制文件和目标代码。

此外，任何可能侵犯泰芯或其他知识产权所有者权利的行为也均被严格禁止。

对于实施上述侵权行为的主体，泰芯有权依据中华人民共和国法律或其他可能适用的法律、国际条约采取必要的法律措施，包括但不限于对侵权人提起诉讼或仲裁，或申请法律强制措施。

珠海泰芯半导体有限公司

2024 年 09 月 24 日



珠海泰芯半导体有限公司  
Zhuhai Taixin Semiconductor Co., Ltd

珠海市高新区港湾一号科创园港 11 栋 3 楼

版权所有 侵权必究

Copyright © 2025 by Tai Xin All rights reserved

|      |            |                   |      |         |
|------|------------|-------------------|------|---------|
| 保密等级 | A          | 泰芯 TXW81x 低功耗开发指南 | 文件编号 | TX-0000 |
| 发行日期 | 2025-04-09 |                   | 文件版本 | V1.0    |

修订记录

| 日期         | 版本   | 描 述  | 修订人      |
|------------|------|------|----------|
| 2025-04-09 | V1.0 | 初始版本 | Hushifei |
|            |      |      |          |
|            |      |      |          |
|            |      |      |          |

|                                                                                                                      |                                                    |                         |
|----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|-------------------------|
| <br>泰芯半导体<br>TaiXin Semiconductor | 珠海泰芯半导体有限公司<br>Zhuhai Taixin Semiconductor Co.,Ltd | 珠海市高新区港湾一号科创园港 11 栋 3 楼 |
|----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|-------------------------|

版权所有 侵权必究

Copyright © 2025 by Tai Xin All rights reserved

|      |            |                   |      |         |
|------|------------|-------------------|------|---------|
| 保密等级 | A          | 泰芯 TXW81x 低功耗开发指南 | 文件编号 | TX-0000 |
| 发行日期 | 2025-04-09 |                   | 文件版本 | V1.0    |

目录

泰芯 TXW81x 低功耗开发指南 ..... 1

1. 概述 ..... 1

2. 休眠模式 ..... 2

3. 休眠与唤醒 ..... 4

    3.1. 进入休眠 ..... 4

    3.2. 唤醒设备 ..... 4

4. 唤醒原因 ..... 7

5. 低功耗开发流程 ..... 9

    5.1. 模式 1 – SYSTEM\_SLEEP\_TYPE\_SRAM\_WIFI ..... 9

        5.1.1. 休眠唤醒流程 ..... 9

        5.1.2. 应用程序挂起和恢复 ..... 11

        5.1.3. RESUME 事件处理 ..... 12

        5.1.4. 单芯片方案 ..... 13

        5.1.5. 带主控方案 ..... 13

    5.2. 模式 2 – SYSTEM\_SLEEP\_TYPE\_WIFI\_DSLEEP ..... 14

        5.2.1. 休眠唤醒流程 ..... 14

    5.3. 模式 3 – SYSTEM\_SLEEP\_TYPE\_SRAM\_ONLY ..... 15

        5.3.1. 休眠唤醒流程 ..... 15

    5.4. 模式 4 – SYSTEM\_SLEEP\_TYPE\_LIGHT\_SLEEP ..... 15

        5.4.1. 休眠唤醒流程 ..... 15

    5.5. 模式 5 – SYSTEM\_SLEEP\_TYPE\_RTCC ..... 16

        5.5.1. 休眠唤醒流程 ..... 16

6. 功耗数据 ..... 17

## 1. 概述

本文是 TXW81x 低功耗的开发指南和使用说明。TXW81x 低功耗支持 2 种类型方案：单芯片方案和带主控方案：

单芯片方案：TXW81x 芯片中实现 WiFi 功能（低功耗），和音视频功能。

带主控方案：TXW81x 芯片中仅实现 WiFi 功能（低功耗），音视频功能在主控端实现。

本文档主要适用于以下工程师：

- 技术支持工程师
- 方案软件开发工程师

本文档适用的产品范围：

| 型号     | 封装 | 包装 |
|--------|----|----|
| TXW81x |    |    |

## 2. 休眠模式

TXW81x SDK 支持以下几种休眠模式，请根据方案需求进行选择：

- **模式 1: SYSTEM\_SLEEP\_TYPE\_SRAM\_WIFI**

休眠时 WiFi 保持连接，芯片所有 SRAM 带电，数据不会丢失，唤醒时系统恢复继续运行。

此休眠模式适用于休眠期间保持 WiFi 连接，同时保持应用程序与云端服务器的连接，支持远程唤醒，且唤醒速度快。

- **模式 2: SYSTEM\_SLEEP\_TYPE\_WIFI\_DSLEEP**

休眠时 WiFi 保持连接，芯片只有前 32K SRAM 带电，其它 SRAM 掉电，唤醒时系统会复位重新初始化运行。此休眠模式的功耗进一步降低，但是唤醒时需要重新启动运行建立连接。

此休眠模式适用于休眠期间保持 WiFi 连接，不需要进行远程保活，对唤醒速度要求不高。

- **模式 3: SYSTEM\_SLEEP\_TYPE\_SRAM\_ONLY**

休眠时 WiFi 断开连接，芯片所有 SRAM 带电，唤醒时系统恢复继续运行。

此休眠模式适用于只支持本地过 IO 或者 timer 唤醒的场景，休眠后 WiFi 断开连接，不支持远程唤醒。

- **模式 4: SYSTEM\_SLEEP\_TYPE\_LIGHT\_SLEEP**

轻度休眠模式，休眠时仅关闭无线收发，CPU 正常运行

此休眠模式适用于暂停无线通信，使芯片处于较低功耗同时又保留外围接口正常工作的状态。此模式需要调用 `system_wakeup` API 唤醒。

- **模式 5: SYSTEM\_SLEEP\_TYPE\_RTCC**

该休眠模式主要实现 RTC 功能，利用 TXW81x 内部的比较器比较器，实时检测电源电压 VCC33 和锂电池的输出电压 VBAT。当 VCC33 小于 VABT 时，系统进入低功耗模式，CPU 停止运行，RTC 根据外部晶振计时，芯片只有 32K SRAM 带电，可以保存数据。当 VCC33 重新上电后，系统退出低功耗模式，系统重新初始化运行。

此休眠模式适用于掉电后用锂电池供电的系统，为了休眠后节省锂电池功耗，只保留 RTC 功能，无 wifi 保活也不保存片内 SRAM 上的数据。

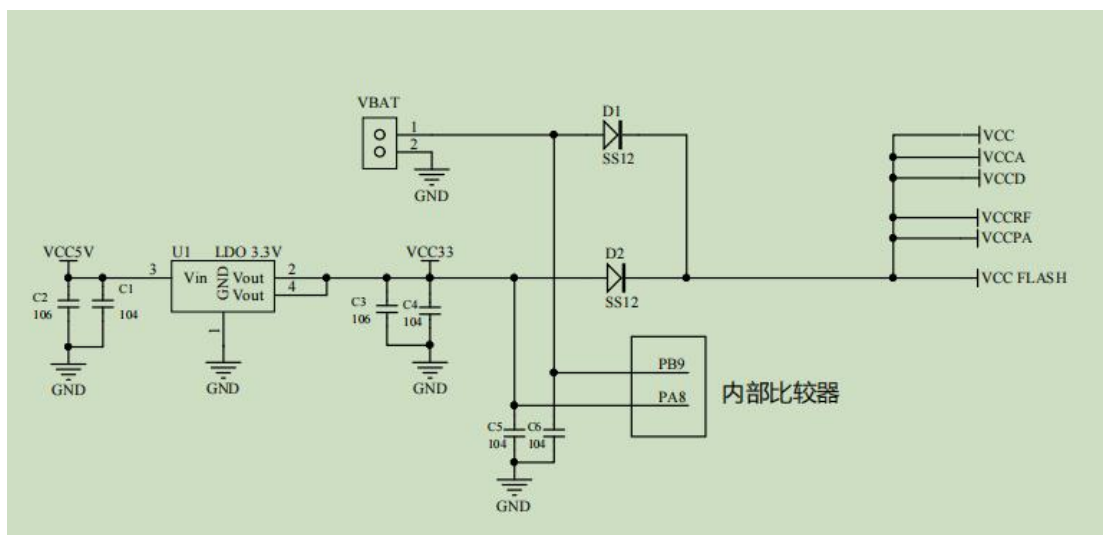
该模式的外围硬件电路要求如下：

- 1) 板上需要外接 32.768KHz 晶振 (LX0SC0)，与 TXW81x 的 PA12 和 PA13 相接：

PA12: LX0SC0 【低速晶振输出】； PA13: LX0SCI 【低速晶振输入】

RTC计时的准确性直接与LX0SC0的精度相关，根据需要选择(如果对精度无要求也可选用内部RC作时钟源)。

- 2) 选择GPIOA的一个引脚作为P端与电源电压VCC33 相接，默认是PA8。
- 3) 选择GPIOB的一个引脚作为N端的参考电压，默认是PB9。



#### ● 模式 6: SHUTDOWN

即关机模式，芯片通过 CHIP\_EN 管脚拉低之后进入 shutdown 模式。当 CHIP\_EN 管脚拉高后芯片退出 shutdown 模式，复位重启然后进入到正常工作模式。

休眠期间 TXW81x 支持在 AP2/PA3/PA8/PA11 上输出 PWM 波形。各种休眠模式的功耗数据见章节“功耗数据”。

## 3. 休眠与唤醒

### 3.1. 进入休眠

TXW81x SDK 进入休眠的 API 只有 1 个：

```
system_sleep(uint16 type, struct system_sleep_param *args)
```

所有休眠模式都使用该 API 进入休眠，只是参数设置不同。

该 API 参数说明：

- type: 设置休眠模式，如[休眠模式](#)所述。
- args: 设置休眠参数，该参数定义如下：

```
struct system_sleep_param {  
    uint32 sleep_ms;  
    uint8  wkup_io_sel[6];    //sel IO0 - IO5  
    uint8  wkup_io_en;        //0: dis; 1 en  
    uint8  wkup_io_edge;      //0: Rising ; 1: Falling  
};
```

- sleep\_ms: 用于设置休眠的时间长度，单位ms，为 0 表示不会产生 timer 唤醒。
- wkup\_io\_sel: 用于选择唤醒的 IO，最多同时支持 6 路 IO 唤醒。
- wkup\_io\_en: 用于使能对应的 IO 唤醒，0 表示关闭对应 IO 唤醒，1 表示打开对应的 IO 唤醒。bit0 对应 wkup\_io\_sel[0]，bit1 对应 wkup\_io\_sel[1]，依次类推。
- wkup\_io\_edge: 用于选择 IO 唤醒的边沿，0 表示上升沿唤醒，1 表示下降沿唤醒，IC 内部会打开响应的下拉或者上拉电阻。bit0 对应 wkup\_io\_sel[0] 的边沿设置，bit1 对应 wkup\_io\_sel[1] 边沿设置，依次类推。

### 3.2. 唤醒设备

TXW81x 的唤醒方式有[远程唤醒](#)和[本地唤醒](#)（IO, timer, API 等）。

- **远程唤醒：**是指 App 远程唤醒，休眠时 WiFi 需要保持连接。App 远程唤醒时 TXW81x

会自动恢复运行或reset重启运行。

- 没有设置唤醒数据的情况下，APP向设备发送任意数据包都会唤醒TXW81x。
- 设置了唤醒数据之后，App发送给设备的数据会被匹配检查，只有检测匹配到是唤醒数据后才会唤醒TXW81x。使用 `sys_sleepdata_request` API设置唤醒数据，示例代码可以参考`sdk/lib/net/utils/udp_kalive.c`中的`udp_kalive_init()`；

```
#if 1
//setup wakeup data.
do{
    struct lmac_wkdata_param *wkdata = sys_sleepdata_request(SYSTEM_SLEEPDATA_ID_WKDATA,
    if(wkdata){
        wkdata->count = LMAC_WKDATA_COUNT;
        wkdata->wkdata_size = LMAC_WKDATA_SIZE;
        wkdata->data[0].sip = udp_kalive.svr.sin_addr.s_addr;
        wkdata->data[0].offset = 20 + 8; //offset from IP hdr
        wkdata->data[0].wkdata_len = os_strlen(wkdata->data[0].wkdata);
        os_memset(wkdata->data[0].mask, 0xff, sizeof(wkdata->data[0].mask));
        os_sprintf(wkdata->data[0].wkdata, "UALIVE:WK-"MACSTR, MAC2STR(local_mac));
        os_printf(KERN_NOTICE"setup wakeup data sip=%u offset= %d len= %d\r\n",
            wkdata->data[0].sip, wkdata->data[0].offset, wkdata->data[0].wkdata_len);
    }
}while(0);
#endif
```

```
#ifndef LMAC_WKDATA_SIZE
#define LMAC_WKDATA_SIZE (64)
#endif
#ifndef LMAC_WKDATA_COUNT
#define LMAC_WKDATA_COUNT (1)
#endif
struct lmac_wkdata_param {
    uint8 count; //must equal to LMAC_WKDATA_COUNT
    uint8 wkdata_size; //must equal to LMAC_WKDATA_SIZE
    uint8 r1, r2;
    struct {
        uint32 sip; //source IP address. can be set 0.
        uint8 offset; //wkdata's offset start from ip hdr.
        uint8 wkdata_len; //wkdata's length
        uint8 r1, r2;
        uint8 mask[LMAC_WKDATA_SIZE / 8]; //match mask.
        uint8 wkdata[LMAC_WKDATA_SIZE]; //wkdata used to match.
    } data[LMAC_WKDATA_COUNT];
};
```

SDK 默认只支持 1 种唤醒数据，且唤醒数据最大长度为 64 bytes。如果默认值无法满足需求，则可以修改 `LMAC_WKDATA_SIZE` 和 `LMAC_WKDATA_COUNT` 宏定义的值。在设置唤醒数据时 `wkdata->count` 和 `wkdata->wkdata_size` 必须设置正确，也就是：

```
wkdata->count=LMAC_WKDATA_COUNT;
```

```
wkdata->wkdata_size=LMAC_WKDATA_SIZE;
```

- **本地唤醒:**

- **I0唤醒:** 由I0触发唤醒, TXW81x支持 6 个唤醒I0, 在执行system\_sleep时进行设置。
- **timer唤醒:** 由内部定时器唤醒, 进入休眠时内部定时器根据设置的sleep\_ms参数进行计数, 休眠时间结束时唤醒芯片。
- **API唤醒:** 只有SYSTEM\_SLEEP\_TYPE\_LIGHT\_SLEEP模式支持API唤醒。
- **比较器唤醒:** TXW81x支持两路比较器唤醒, 可以在休眠中设置两个I0作为比较器输入的N端和P端, 当比较器输出位 0 或者 1 时产生唤醒信号。

## 4. 唤醒原因

TXW81x 被唤醒时，SDK 会提供唤醒原因以供查询。调用 `sys_wakeup_reason` API 可以查询当前被唤醒的原因。

```
uint8 sys_wakeup_reason(void);
```

当前定义的唤醒原因主要有以下几种：

```
typedef enum {  
  
    //timer 唤醒  
  
    DSLEEP_WK_REASON_TIMER            = 1,  
  
    //AP Beacon TIM 标识唤醒 - 单播数据唤醒，通常是远程唤醒  
  
    DSLEEP_WK_REASON_TIM              = 2,  
  
    //AP Beacon Bcast TIM 标识唤醒 - 广播数据唤醒  
  
    DSLEEP_WK_REASON_BC_TIM          = 3,  
  
    //IO 唤醒  
  
    DSLEEP_WK_REASON_IO              = 4,  
  
    //由于 AP Beacon 丢失产生的唤醒  
  
    DSLEEP_WK_REASON_BEACON_LOST     = 5,  
  
    //检测到 AP 异常产生的唤醒，通常是 AP 发生了重启  
  
    DSLEEP_WK_REASON_AP_ERROR        = 6,  
  
    //远程心跳超时  
  
    DSLEEP_WK_REASON_HB_TIMEOUT      = 7,  
  
    //唤醒数据匹配唤醒  
  
    DSLEEP_WK_REASON_WK_DATA         = 8,  
  
    //芯片 MLCR IO  
  
    DSLEEP_WK_REASON_MCLR            = 9,
```

```

//低电复位

DSLEEP_WK_REASON_LVD          = 10,

//PIR 检测唤醒

DSLEEP_WK_REASON_PIR          = 11,

//由于 AP 执行了唤醒命令产生的唤醒

DSLEEP_WK_REASON_APWK        = 12,


//AP 检测到 STA 休眠时断开连接

DSLEEP_WK_REASON_PS_DISCONNECT = 13,

//STANDBY 模式唤醒

DSLEEP_WK_REASON_STANDBY      = 14,

//由于 AP 检测到 STA 保活超时产生的唤醒

DSLEEP_WK_REASON_MAX_IDLE_TMO = 15,


DSLEEP_WK_REASON_STA_ERROR    = 20,

DSLEEP_WK_REASON_SLEPED_STA_ERROR= 21,


//AP 休眠时被 STA 唤醒

DSLEEP_WK_REASON_STA_DATA     = 22,


DSLEEP_WK_REASON_AP_PAIR      = 23,

} DSLEEP_WK_REASON;

```

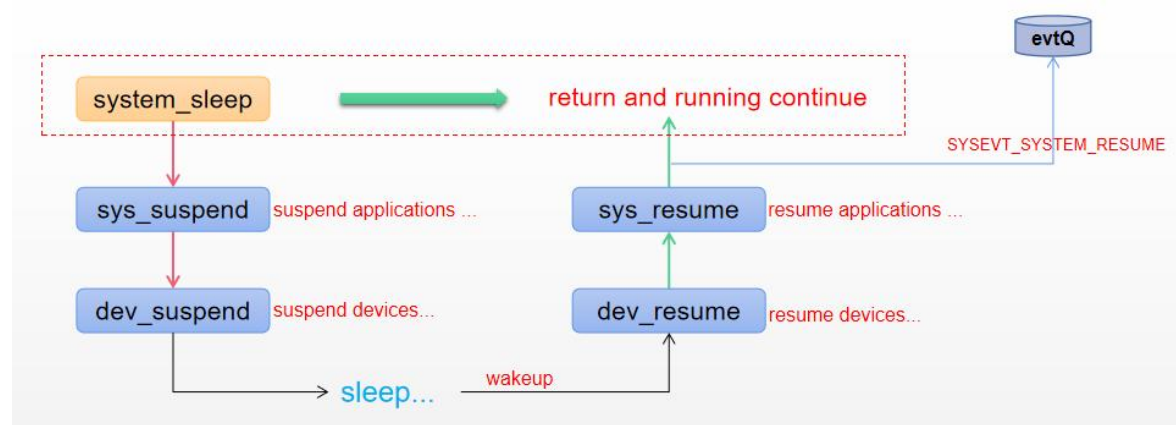
## 5. 低功耗开发流程

根据实际方案需求选择休眠模式，不同休眠模式的开发流程有所不同，本章节将介绍各种休眠模式的开发流程。[请仔细阅读模式 1 的休眠流程，其它休眠模式与模式 1 类似部分省略了描述。](#)

### 5.1. 模式 1 - SYSTEM\_SLEEP\_TYPE\_SRAM\_WIFI

#### 5.1.1. 休眠唤醒流程

休眠时 WiFi 保活，而且支持应用程序保活；唤醒时系统恢复继续运行。休眠唤醒流程如下图所示：



如上图所示，在休眠过程中会先挂起应用程序，再挂起设备；唤醒过程中会先恢复设备，再恢复应用程序，同时产生一个 event：SYSEVT\_SYSTEM\_RESUME。

- **sys\_suspend**：挂起应用程序。

在系统进入休眠前，首先执行 sys\_suspend 挂起部分应用程序。受休眠影响的应用程序都需要执行挂起操作，通常是一些网络相关功能模块，进入休眠前停止数据发送。

需要执行挂起的应用代码模块在初始化时使用 sys\_register\_sleepcb 函数进行注册，在休眠流程中执行 sys\_suspend 操作时会回调执行注册的回调函数。在回调函数中执行一些挂起操作。

- **dev\_suspend:** 挂起设备

在休眠过程的最后阶段会执行 dev\_suspend 挂起设备，备份设备状态信息以便于系统唤醒时能够恢复设备状态。SDK 提供的驱动代码都已经支持 suspend 和 resume 操作，如果自行添加的外设驱动代码，则需要自行实现 suspend 和 resume 操作。

另外 SDK 提供了 **dev\_suspend\_hook** 接口，在每个 device 被执行 suspend 之前会回调该函数，可以在该函数中对设备进行特殊处理。该函数为 weak 函数，需要使用该函数时请重写该函数。该函数返回 1，表示该 device 继续执行 suspend 操作，返回 0 则终止该设备的 suspend 操作。

```
__weak int32 dev_suspend_hook(struct dev_obj *dev, uint16 type)
{
    return 1;
}
```

- **dev\_resume:** 恢复设备

系统被唤醒时先执行恢复设备操作，将各个设备恢复到休眠前的状态。

SDK 提供了 **dev\_resume\_hook** 接口，在每个 device 执行 resume 之前会回调该函数，可以在该函数中对设备进行特殊处理。例如根据唤醒原因决定该 device 是否执行 resume 操作。该函数为 weak 函数，需要使用该函数时请重写该函数。该函数返回 1，表示该 device 继续执行 resume 操作，返回 0 则终止该设备的 resume 操作。

```
33:
34: int32 dev_resume_hook(struct dev_obj *dev, uint16 type, uint32 wkreason)
35: {
36:     if (wkreason == DSLEEP_WK_REASON_TIMER) {
37:         if (dev->dev_id == HG_SDIO_SLAVE_DEVID) {
38:             return 0; ← return 0, this device needn't resume
39:         }
40:     }
41:     return 1; ← return 1, this device resume continue.
42: }
43:
44: #endif
45:
```

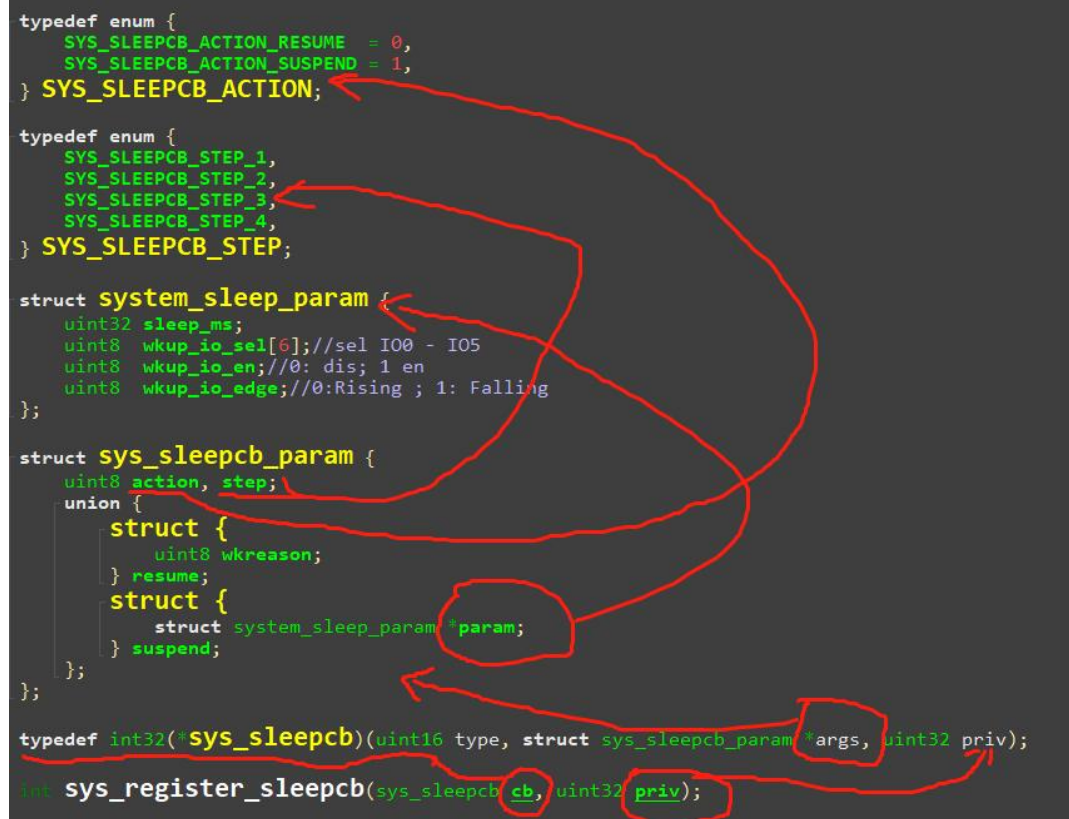
- **sys\_resume:** 恢复应用程序

设备恢复之后执行恢复应用程序，回调应用程序所注册的回调函数，恢复应用程序运行。

### 5.1.2. 应用程序挂起和恢复

如上所述，在低功耗开发中需要关注的就是应用程序的挂起和恢复，以及 `SYSEVT_SYSTEM_RESUME` 消息的处理。

应用程序的挂起与恢复，需要向系统注册 sleepcb 回调。相关 API 定义如下：



```
typedef enum {
    SYS_SLEEP_CB_ACTION_RESUME = 0,
    SYS_SLEEP_CB_ACTION_SUSPEND = 1,
} SYS_SLEEP_CB_ACTION;

typedef enum {
    SYS_SLEEP_CB_STEP_1,
    SYS_SLEEP_CB_STEP_2,
    SYS_SLEEP_CB_STEP_3,
    SYS_SLEEP_CB_STEP_4,
} SYS_SLEEP_CB_STEP;

struct system_sleep_param {
    uint32 sleep_ms;
    uint8 wkup_io_sel[6]; //sel IO0 - IO5
    uint8 wkup_io_en; //0: dis; 1: en
    uint8 wkup_io_edge; //0: Rising; 1: Falling
};

struct sys_sleepcb_param {
    uint8 action, step;
    union {
        struct {
            uint8 wkreason;
        } resume;
        struct {
            struct system_sleep_param *param;
        } suspend;
    };
};

typedef int32_t(*sys_sleepcb)(uint16_t type, struct sys_sleepcb_param *args, uint32_t priv);

int sys_register_sleepcb(sys_sleepcb cb, uint32_t priv);
```

The image shows a screenshot of C code defining the sleepcb API. Red annotations highlight key parts: a red arrow points from the `SYS_SLEEP_CB_ACTION` enum to the `action` field in `sys_sleepcb_param`; another red arrow points from the `SYS_SLEEP_CB_STEP` enum to the `step` field in `sys_sleepcb_param`; a red circle highlights the `param` pointer in the `suspend` union member; and red circles highlight the `cb` and `priv` parameters in the `sys_register_sleepcb` function signature.

回调函数类型 `sys_sleepcb`:

- 参数 `type`: 执行 `system_sleep` API 时设置的休眠模式。
- 参数 `args`: 挂起或恢复应用程序的参数

```
struct sys_sleepcb_param {
    uint8 action, step;
    union {
        struct {
            uint8 wkreason;
        } resume;
        struct {
            struct system_sleep_param *param;
        } suspend;
    };
};
```

```

        } suspend;

    };

};

```

- 参数 priv: 注册 sleepcb 时的输入参数 priv, 回调时会传回该参数。

应用程序的挂起与恢复定义了 4 个优先级, 挂起时按 STEP1/2/3/4 的顺序执行, 恢复时按 STEP4/3/2/1 的顺序执行。

如果某个应用模块需要区分执行优先级则可以在回调函数中判断 step 值, 以区分执行。通常应用程序在 step1 中执行就可以了, step4 默认用来执行 SDK 内部协议栈模块的挂起。

```

typedef enum {
    SYS_SLEEP_CB_STEP_1,
    SYS_SLEEP_CB_STEP_2,
    SYS_SLEEP_CB_STEP_3,
    SYS_SLEEP_CB_STEP_4,
} SYS_SLEEP_CB_STEP;

```

### 5.1.3. RESUME 事件处理

芯片被唤醒时会产生 SYSEVT\_SYSTEM\_RESUME 事件, 需要开发代码处理该事件。查询当前唤醒原因, 根据唤醒原因执行相应的代码。

SDK 默认已添加了该事件的处理代码, 如下:

```

23:
24: sysevt_hdl_res sys_event_hdl(uint32 event_id, uint32 data, uint32 priv)
25: {
26:     struct netif *nif;
27:
28:     switch (event_id) {
29:         case SYS_EVENT(SYS_EVENT_SYSTEM, SYSEVT_SYSTEM_RESUME):
30:             #ifdef CONFIG_SUSPEND
31:                 sys_check_wkreason(WKREASON_RESUME);
32:             #endif
33:             break;
34:
35:         case SYS_EVENT(SYS_EVENT_WIFI, SYSEVT_WIFI_CONNECTED):
36:             sys_status.wifi_connected = 1;
37:             #if WIFI_STA_SUPPORT && SYS_NETWORK_SUPPORT
38:                 if (sys_cfgs.wifi_mode == WIFI_MODE_STA && sys_cfgs.dhpc_en) {
39:                     sys_status.dhpc_done = 0;
40:                     lwip_netif_set_dhcp2("w0", 1);
41:                 }
42:             #endif

```

在 sys\_check\_wkreason 函数中查询唤醒原因，执行相应的代码。

```
void sys_check_wkreason(uint8 type)
{
    uint8 i;
    uint8 wk = 0;
    uint8 wkreason = sys_wakeup_reason();

    os_printf(KERN_NOTICE "wakeup reason: %d\r\n", wkreason);
    if (wkreason == 0) {
        return;
    }

    for (i = 0; i < sys_cfgs.wkhost_reasons; i++) {
        if (sys_cfgs.wkhost_reasons[i] == wkreason) {
            wk = 1;
            break;
        }
    }

    switch (wkreason) {
        case DSLEEP_WK_REASON_TIMER:
            wk = 0;
            udp_kalive_start(); //Keepalive sample code.
            break;
        case DSLEEP_WK_REASON_TIM:
            break;
        case DSLEEP_WK_REASON_BC_TIM:
            break;
        case DSLEEP_WK_REASON_IO:
            break;
        case DSLEEP_WK_REASON_BEACON_LOST:
            break;
        case DSLEEP_WK_REASON_AP_ERROR:
            break;
        case DSLEEP_WK_REASON_HB_TIMEOUT:
            break;
        case DSLEEP_WK_REASON_WK_DATA:
            break;
        case DSLEEP_WK_REASON_MCLR:
            break;
        case DSLEEP_WK_REASON_LVD:
            break;
        case DSLEEP_WK_REASON_PIR:
            break;
        case DSLEEP_WK_REASON_APWK:
            break;
        default:
            break;
    } // end switch wkreason

    if (wk) {
        sys_wakeup_host();
    }
}
```

get current wakeup reason

wake reason is NULL, return.

check the wake reason if need to wakeup host.

TIMER wakeup, just do keepalive needn't wakeup host.

wakeup host

示例中的 udp\_kalive\_start() 函数是 SDK 提供的 udp 保活示例代码，源码位于 sdk/lib/net/utils/udp\_kalive.c。可参考该示例代码，自行开发所需的保活代码。

udp\_kalive.c 所实现的功能为：在休眠期间定时醒来向服务器发送保活包，报告设备的状态，保活包发送失败 10 次时则认为链路断开，唤醒主控。

#### 5.1.4. 单芯片方案

单芯片方案使用 WiFi 芯片内部的音视频功能，使用 FPV SDK 进行二次开发。

#### 5.1.5. 带主控方案

在带主控的方案中，主要的业务逻辑均在主控端执行，WiFi 模块内部可以执行与服务

器的保活行为。

WiFi 驱动会在主控端创建 wlan0 接口,同时 WiFi 模块内部包含了完整的 TCPIP 协议栈,也会创建 w0 接口。设备运行时主控 wlan0 接口和 WiFi 内部 w0 接口,具有相同的 MAC 地址和相同的 IP 地址,各自运行不同的应用程序。WiFi 模块会识别内部产生的 tcp/udp 数据流,以判断是 WiFi 模块内部数据流还是主控数据流。WiFi 模块默认只支持识别 8 路数据流,所以 WiFi 模块内部的应用程序应控制 tcp/udp 数据流的数量。WiFi 模块打印“NO FREE LOCALID”时表明内部数据流已经超过限值。

可以设置由 WiFi 模块提前 dhcp 请求 IP 地址,主控开机后直接使用 WiFi 模块申请的 IP 地址,节省 dhcp 时间。

当主控通知 WiFi 模块休眠时, WiFi 模块执行休眠流程进入休眠状态。被唤醒时检查唤醒决定是否唤醒主控。主控被唤醒时应主动查询唤醒原因,执行相应的程序。

## 5.2. 模式 2 - SYSTEM\_SLEEP\_TYPE\_WIFI\_DSLEEP

在模式 2 时只有芯片前 32K SRAM 带电,功耗比模式 1 进一步降低。进入休眠时 WiFi 保持连接,支持远程唤醒,但是不支持应用程序保活。唤醒时芯片会复位重新初始化运行,需要重新建立 WiFi 连接。

### 5.2.1. 休眠唤醒流程

由于休眠期间不需要应用程序保活,而且唤醒时芯片会复位重新初始化运行,所以休眠流程不需要执行 suspend 操作。WiFi 模块收到休眠指令时,直接进入休眠状态。应用程序数据与状态则直接丢失,唤醒时芯片会复位重新初始化运行。

32K SRAM 中有 4K 区域可以用于保存应用数据,芯片复位时该 4K 区域数据**保持不变**。所以应用程序利用该区域存储自定义数据,在系统初始化时读取该区域的数据使用,同时唤醒原因在芯片复位时也保持不变,应用程序可以查询唤醒原因进行相应的处理。

### 5.3. 模式 3 - SYSTEM\_SLEEP\_TYPE\_SRAM\_ONLY

在模式 3 休眠时，芯片所有的 SRAM 带电，WiFi 功能关闭；唤醒时自动打开 WiFi 功能，系统恢复运行，应用程序继续运行。此模式适用于休眠时不需要 WiFi 功能的方案。

#### 5.3.1. 休眠唤醒流程

此休眠模式的休眠唤醒流程与模式 1 类似，区别在于休眠时 WiFi 功能被关闭。低功耗开发流程与模式 1 相同。

### 5.4. 模式 4 - SYSTEM\_SLEEP\_TYPE\_LIGHT\_SLEEP

此休眠模式为轻度休眠，不支持 WiFi 保活。休眠时 watchdog 和 CPU 仍然在低速运行，可以执行部分简单的用户程序，比如点亮 LED，UART 数据收发等。

#### 5.4.1. 休眠唤醒流程

此休眠模式的休眠流程与模式 1 类似，进入休眠时会执行 device 和应用模块的挂起操作。默认所有的 device 都会被挂起，可以在 dev\_suspend\_hook 函数中保留需要工作的 device。例如需要在休眠期间使用 uart，则可以在 dev\_suspend\_hook 中终止 uart 的 suspend，同时重新设置 uart 波特率等参数。应用模块也可以类似处理，对不需要挂起的应用模块可以在 sleepcb 中特殊处理，或者不注册 sleepcb。

此休眠模式的唤醒，需要执行 system\_wakeup API。唤醒时需要重新配置系统时钟，以恢复系统正常运行，WiFi 功能自动打开重新建立连接。

## 5.5. 模式 5 - SYSTEM\_SLEEP\_TYPE\_RTCC

### 5.5.1. 休眠唤醒流程

调用 `dsleep_rtc_init(uint8 PA_n, uint8 PB_n)` API 初始初始化 RTC 模块, 其中 PA\_n 可选范围是 PA\_0~PA\_14, PB\_n 可选范围是 PB\_0~PB\_14。当检测到电源当 VCC33 小于 VABT 时, 模组开始进入模式 5 休眠, 会有打印信息 “sys\_sleep...”

当检测到 VCC33 电压恢复正常后, 模组退出模式 5 休眠, 打印 “soft\_reset”, 通过 `dsleep_rtc_get()` API 可以获取当前的 RTC 值, 返回值类型是 uint32, 每 1/8 秒增加 1。

## 6. 功耗数据

以下测试在环境温度 25℃，STA 打开 RX 接收 AP Beacons 时间为 1ms、测试电压 3.3V、单芯片方案，屏蔽环境下测试。

| v200             | DTIM1 | DTIM3 | DTIM5 | DTIM6 | DTIM10 | DTIM30 | 单位 |
|------------------|-------|-------|-------|-------|--------|--------|----|
| 内置 32K<br>, DCDC | 2.277 | 0.804 | 0.509 | 0.435 | 0.288  | 0.216  | mA |
| 外置 32K<br>, DCDC | 2.273 | 0.792 | 0.490 | 0.413 | 0.251  | 0.178  | mA |

说明：使用内部 32K 晶振时，休眠时间越长会带来越大的误差，所以会导致功耗有所上升。